

DIRECTION NATIONALE DE L'ENSEIGNEMENT
SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE



Ecole Nationale d'Ingénieurs - Abderrahmane Baba Touré

410, Av. Van Vollenhoven BP 242 – Tél : (223) 20 22 27 36 – Fax : (223) 20 21 50 38 / Bamako
– MALI.

Département Génie Informatique & Télécommunications

Application de gestion des employés : Gesem

Master GIT-S3

Filière : MSI

Matière	Ingénierie des SI à base de composants	
Nom et Prénom du tuteur	Dr. Yacouba GOITA	
Noms et Prénoms	Drissa Sidiki TRAORE Souleymane COULIBALY Bagaoussou COULIBALY Fatoumata DIAKITE Pascal DIARRA	
Date : 15/11/2025	Groupe : N°02	Durée :
Dépôt : 08/11/2025		

NOTE	APPRECIATION
------	--------------

..... /20	
-----------	--

ANNEE : 2024-2025

Table des matières

Introduction.....	3
Contexte du projet.....	3
Objectifs.....	3
Structure du rapport.....	3
Analyse des Besoins et Conception.....	4
Fonctionnalités requises.....	4
Authentification.....	4
Gestion des Employés.....	4
Modèle Conceptuel de Données (MCD).....	4
Script SQL de création des tables	5
Architecture logicielle.....	6
Séparation des responsabilités.....	8
Technologies et Outils Utilisés.....	8
Implémentation.....	9
Structure du projet.....	9
Détails d'implémentation	10
Authentification.....	10
Gestion des Employés.....	11
Persistance avec JPA	11
Sécurité.....	12
Filtre d'authentification	12
Gestion de session.....	13
Captures d'écran de l'interface.....	13
Conclusion	16
Bilan du projet	16
Perspectives d'évolution.....	17
Apports personnels	17

Introduction

Contexte du projet

Le projet **Gesem** (Gestion des Employés) a été développé dans le cadre du module **Ingénierie des SI à base de composants**. Ce projet a pour objectif pédagogique d'illustrer et de mettre en pratique les concepts fondamentaux de **Jakarta EE** (anciennement Java EE), notamment :

- Les **Servlets** pour la gestion des requêtes HTTP
- Les **JSP** (JavaServer Pages) pour la génération dynamique de vues
- **JPA** (Java Persistence API) pour la persistance des données
- L'architecture **MVC** (Modèle-Vue-Contrôleur)
- La gestion des sessions et la sécurité

L'application Gesem est une application web de gestion des employés pour une entreprise permettant aux administrateurs authentifiés de **consulter**, **ajouter**, **modifier** et **supprimer** des informations sur leur employés.

Objectifs

Les objectifs principaux du projet, étaient les suivants :

1. **Authentification** : Mise en place d'un système de connexion sécurisé avec gestion de session
2. **Gestion CRUD des employés** : Implémentation complète des opérations **Create**, **Read**, **Update**, **Delete**
3. **Architecture MVC** : Respect d'une architecture propre avec séparation des responsabilités
4. **Persistance des données** : Utilisation de JPA avec PostgreSQL
5. **Sécurité** : Protection des ressources par filtres de sécurité

Tous ces objectifs ont été atteints avec succès. L'application permet une gestion complète des employés avec authentification, et respecte les bonnes pratiques de développement Java EE.

Structure du rapport

Ce rapport est organisé en plusieurs sections :

- **Section 1** : Analyse des besoins et conception, incluant le modèle de données et l'architecture
- **Section 2** : Présentation de l'architecture MVC utilisée
- **Section 3** : Technologies et outils
- **Section 4** : Détails d'implémentation avec extraits de code
- **Section 5** : Capture d'écran de l'interface
- **Section 6** : Conclusion

Analyse des Besoins et Conception

Fonctionnalités requises

Authentification

L'application doit permettre aux administrateur (Utilisateur avec le rôle **admin**) de se connecter avec un **login** et un **mot de passe**. Les fonctionnalités implémentées sont :

- **Page de connexion** : Formulaire de login accessible à l'URL */Login*
- **Vérification des identifiants** : Validation des crédenciales (login + mot de passe) dans la base de données
- **Gestion de session** : Création d'une session HTTP après authentification réussie
- **Déconnexion** : Invalidation de la session et redirection vers la page de login
- **Protection des ressources** : Filtre de sécurité redirigeant les utilisateurs non authentifiés

Flux d'authentification :

Gestion des Employés

L'application offre un système complet de gestion CRUD pour les employés :

- **Liste des employés** : Affichage de tous les employés dans un tableau avec leurs informations
- **Ajout d'un employé** : Formulaire permettant de créer un nouvel employé avec validation
- **Modification d'un employé** : Édition des informations d'un employé existant
- **Suppression d'un employé** : Suppression avec confirmation

Flux CRUD :

Modèle Conceptuel de Données (MCD)

Le modèle de données comprend deux entités principales :

Entité **User** (Utilisateur)

- id : UUID (identifiant unique, clé primaire)
- login : String (unique, non null)
- password : String (non null)
- role : String (non null)

Entité **Employe** (Employé)

- id : UUID (identifiant unique, clé primaire)
- nom : String (non null)
- prenom : String (non null)
- poste : String (non null)
- email : String (unique, non null)
- dateEmbauche : LocalDate (non null)
- salaire : BigDecimal (non null, précision 10, échelle 2)
- typeContrat : TypeContrat (enum : CDD ou CDI, non null)

Relations : Aucune relation directe entre User et Employe (architecture simple). Chaque entité est indépendante.

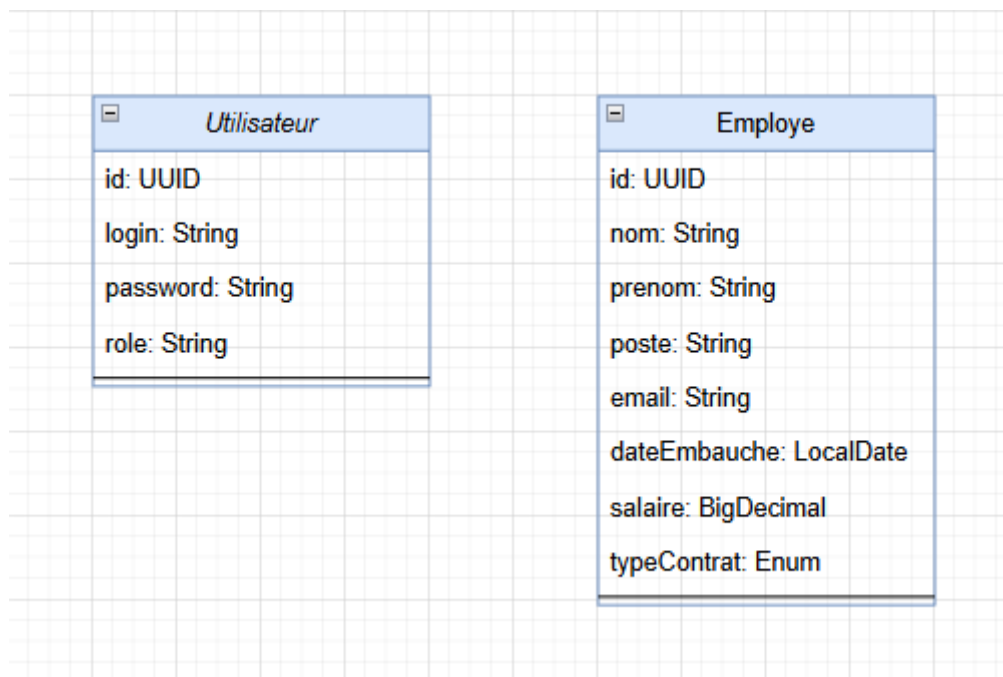


Figure 1: Model Conceptuel de Données (MCD)

Script SQL de création des tables

Table utilisateur

```
CREATE TABLE utilisateur (  
    id UUID PRIMARY KEY,  
    login VARCHAR(255) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    role VARCHAR(255) NOT NULL  
);
```

Justifications :

- UUID pour l'identifiant : évite les problèmes de concurrence et permet la génération côté client.
- Contrainte UNIQUE sur login : garantit l'unicité des identifiants de connexion.
- Pas de hashage du mot de passe dans cette version.

Table employe

```
CREATE TABLE employe (  
  id UUID PRIMARY KEY,  
  nom VARCHAR(255) NOT NULL,  
  prenom VARCHAR(255) NOT NULL,  
  poste VARCHAR(255) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  date_embauche DATE NOT NULL,  
  salaire NUMERIC(10, 2) NOT NULL,  
  type_contrat VARCHAR(10) NOT NULL CHECK (type_contrat IN ('CDD', 'CDI'))  
);
```

Justifications :

- UUID pour l'identifiant : cohérence avec la table utilisateur
- Contrainte UNIQUE sur email : un employé = un email unique
- Type NUMERIC pour le salaire : précision monétaire (10 chiffres, 2 décimales)
- CHECK constraint sur type_contrat : validation au niveau base de données

Génération automatique : Les tables sont créées automatiquement par JPA grâce à la configuration ***schema-generation.database.action=create-or-extend-tables*** dans ***persistence.xml***.

Architecture logicielle

L'application suit une architecture **MVC** (Modèle-Vue-Contrôleur).

L'architecture MVC (Modèle-Vue-Contrôleur) a été choisie pour structurer l'application de manière claire et maintenable.

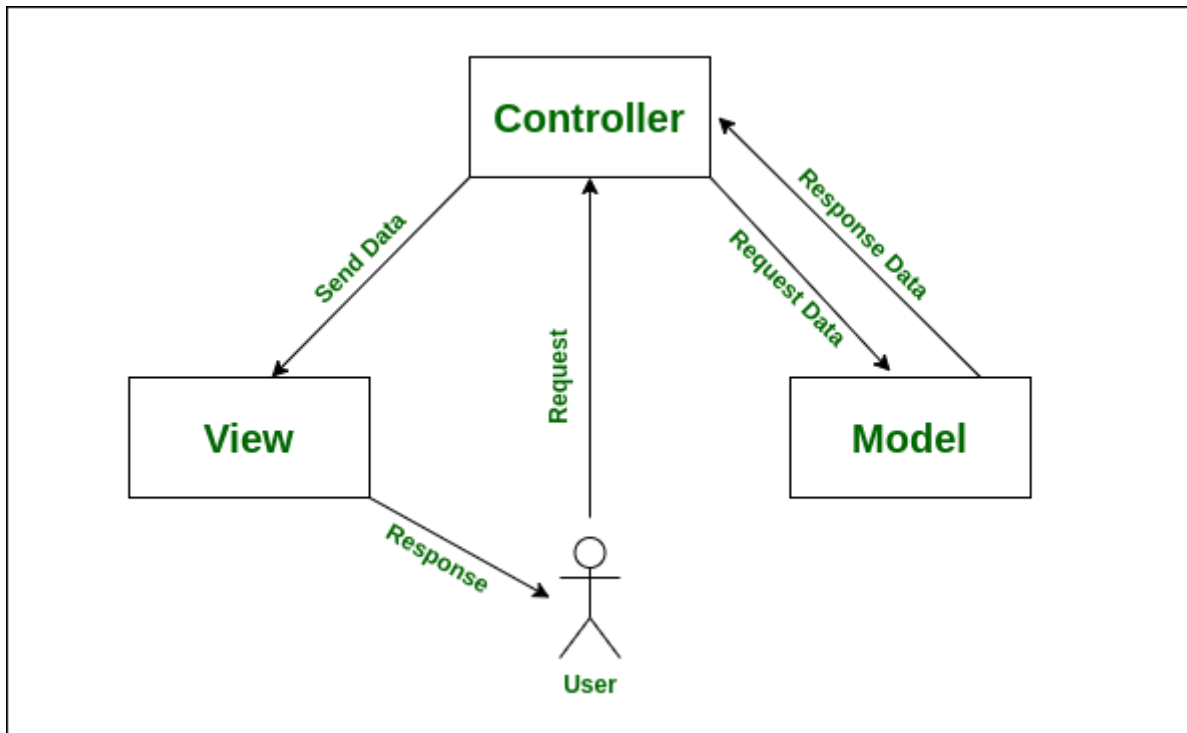


Figure 2: Architecture MVC

Modèle (Model)

- **Entités JPA** : `User.java`, `Employe.java`, `TypeContrat.java`
- **DAOs** : `UserDAO.java`, `EmployeDAO.java` (couche d'accès aux données)
- **Utilitaires** : `JPAUtil.java` (gestion de l'`EntityManager`)

Vue (View)

- **JSP** dans `WEB-INF/views/` :
 - `login.jsp` : Page de connexion
 - `employes/liste.jsp` : Liste des employés
 - `employes/formulaire.jsp` : Formulaire d'ajout/modification
- **CSS** : Feuilles de style dans `/css/`

Contrôleur (Controller)

- **Servlets** :
 - `LoginServlet` : Gestion de l'authentification
 - `LogoutServlet` : Gestion de la déconnexion
 - `EmployeServlet` : Gestion CRUD des employés
- **Filtres** :
 - `AuthFilter` : Protection des ressources

Avantages de cette architecture :

- **Séparation des responsabilités** : Chaque couche a un rôle bien défini
- **Maintenabilité** : Modifications isolées par couche
- **Testabilité** : Possibilité de tester chaque composant indépendamment
- **Réutilisabilité** : Les DAOs peuvent être réutilisés dans d'autres contextes

Séparation des responsabilités

Chaque composant a une responsabilité unique :

Composant	Responsabilité
Entités JPA	Représentation des données
DAOs	Accès et manipulation des données
Servlets	Traitement des requêtes http
JSP	Génération de la présentation
Filtres	Sécurité et pré-traitement

Technologies et Outils Utilisés

Pour la réalisation de ce mini projet, voici les technologies et outils qui ont été utilisés :

Composant	Technologie retenue	Version	Justification
Langage principal	Java	11+	Standard pour Jakarta EE, compatibilité
Framework Web	Jakarta EE (Jakarta EE)	9.1.0	Standard industriel, pas de dépendance externe
Servlets	Jakarta Servlet API	5.0	Gestion native des requêtes HTTP
Vues	JSP (Jakarta Server Pages)	3.0	Génération dynamique de pages HTML
Persistence	JPA (Jakarta Persistence)	3.0	Standard ORM, indépendance du SGBD

Implémentation JPA	EclipseLink	4.0.1	Implémentation de référence, performante
Base de données	PostgreSQL	12+	SGBD relationnel robuste et open-source
Pilote JDBC	PostgreSQL JDBC Driver	42.6.0	Connexion à PostgreSQL
Build Tool	Maven	3.6+	Gestion des dépendances et build automatisé
Serveur d'application	Tomcat / GlassFish	10+	Compatible Jakarta EE 9.1
SGBD	PostgreSQL	17	Simplicité

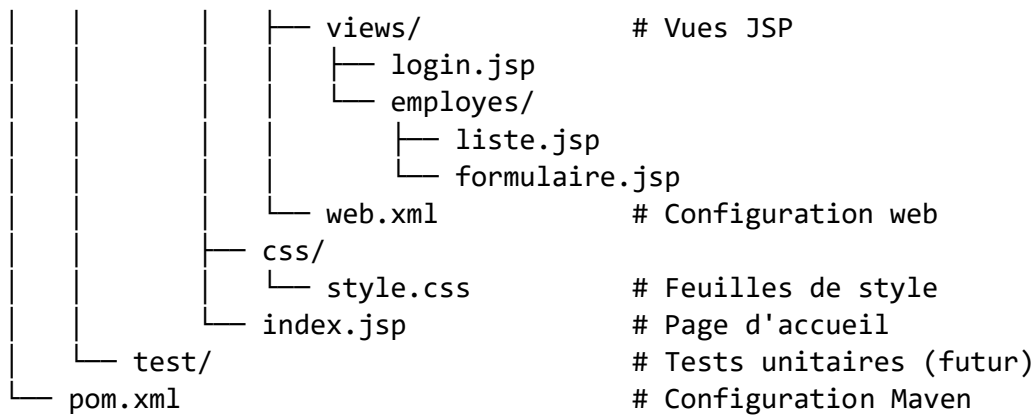
Implémentation

Structure du projet

```

gesem/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/kelenpe/eni/m2/gesem/
│   │   │   │   ├── model/                # Entités JPA
│   │   │   │   │   ├── User.java
│   │   │   │   │   ├── Employe.java
│   │   │   │   │   └── TypeContrat.java
│   │   │   │   ├── dao/                  # Data Access Objects
│   │   │   │   │   ├── UserDAO.java
│   │   │   │   │   └── EmployeDAO.java
│   │   │   │   ├── controller/          # Servlets
│   │   │   │   │   ├── LoginServlet.java
│   │   │   │   │   ├── LogoutServlet.java
│   │   │   │   │   └── EmployeServlet.java
│   │   │   │   ├── filter/              # Filtres
│   │   │   │   │   └── AuthFilter.java
│   │   │   │   ├── util/                # Utilitaires
│   │   │   │   │   └── JPAUtil.java
│   │   │   └── resources/
│   │   │       ├── META-INF/
│   │   │       │   ├── persistence.xml    # Configuration JPA
│   │   │       │   └── beans.xml
│   │   │       └── init-db.sql            # Script d'initialisation
│   │   └── webapp/
│   │       └── WEB-INF/

```



Détails d'implémentation

Authentification

LoginServlet gère l'authentification :

```

@WebServlet(name = "LoginServlet", value = "/login")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response) {
        String login = request.getParameter("login");
        String password = request.getParameter("password");

        UserDao userDao = new UserDao();
        User user = userDao.findByLoginAndPassword(login, password);

        if (user != null) {
            HttpSession session = request.getSession();
            session.setAttribute("user", user);
            response.sendRedirect("/employees");
        } else {
            request.setAttribute("error", "Identifiants incorrects");
            // Afficher le formulaire avec erreur
        }
    }
}

```

Fonctionnement :

1. Récupération des paramètres login et password
2. Vérification dans la base de données via UserDao
3. Si valide : création de session et redirection
4. Si invalide : affichage d'un message d'erreur

Gestion des Employés

EmployeServlet gère toutes les opérations CRUD via un paramètre action :

- action=null ou vide : Liste des employés
- action=add : Affichage du formulaire d'ajout
- action=edit&id=xxx : Affichage du formulaire de modification
- action=delete&id=xxx : Suppression d'un employé

Exemple d'ajout :

```
if ("add".equals(action)) {  
    // Validation des champs  
    // Vérification email unique  
    Employe employe = new Employe(nom, prenom, poste, email, dateEmbauche,  
salaire, typeContrat);  
    employeDAO.create(employe);  
    request.setAttribute("success", "Employé ajouté avec succès");  
}
```

Validation :

- Tous les champs sont obligatoires
- Email doit être unique - Salaire doit être positif
- Type de contrat doit être CDD ou CDI
- Date d'embauche au format yyyy-MM-dd

Persistence avec JPA

Configuration persistence.xml :

```
<persistence-unit name="gesem-pu" transaction-type="RESOURCE_LOCAL">  
    <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>  
    <class>com.kelenpe.eni.m2.gesem.model.User</class>  
    <class>com.kelenpe.eni.m2.gesem.model.Employe</class>  
    <properties>  
        <property name="jakarta.persistence.jdbc.url"  
            value="jdbc:postgresql://localhost:5432/gesem_db"/>  
        <property name="jakarta.persistence.jdbc.user" value="postgres"/>  
        <property name="jakarta.persistence.jdbc.password" value="votre_mot_d  
e_passe"/>  
        <property name="jakarta.persistence.schema-generation.database.acti  
on"  
            value="create-or-extend-tables"/>  
    </properties>  
</persistence-unit>
```

Utilisation dans les DAOs :

```
public class EmployeDAO {  
    private EntityManager entityManager;
```

```

public EmployeeDAO() {
    this.entityManager = JPAUtil.getEntityManager();
}

public void create(Employe employe) {
    entityManager.getTransaction().begin();
    try {
        entityManager.persist(employe);
        entityManager.getTransaction().commit();
    } catch (Exception e) {
        entityManager.getTransaction().rollback();
        throw e;
    }
}
}

```

Sécurité

Filtre d'authentification

AuthFilter protège toutes les URLs /employes :

```

@WebFilter(filterName = "AuthFilter", urlPatterns = {"/employes", "/employes/*"})
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain) {
        HttpSession session = httpRequest.getSession(false);
        boolean isLoggedIn = (session != null && session.getAttribute("user") != null);

        if (!isLoggedIn) {
            httpResponse.sendRedirect("/login");
            return;
        }

        chain.doFilter(request, response);
    }
}

```

Fonctionnement :

- Intercepte toutes les requêtes vers /employes
- Vérifie la présence d'une session valide
- Redirige vers /login si non authentifié
- Laisse passer si authentifié

Gestion de session

- **Création** : Lors de l'authentification réussie
- **Timeout** : 30 minutes d'inactivité (configuré dans web.xml)
- **Invalidation** : Lors de la déconnexion ou expiration
- **Stockage** : Objet User et userLogin dans la session

Captures d'écran de l'interface

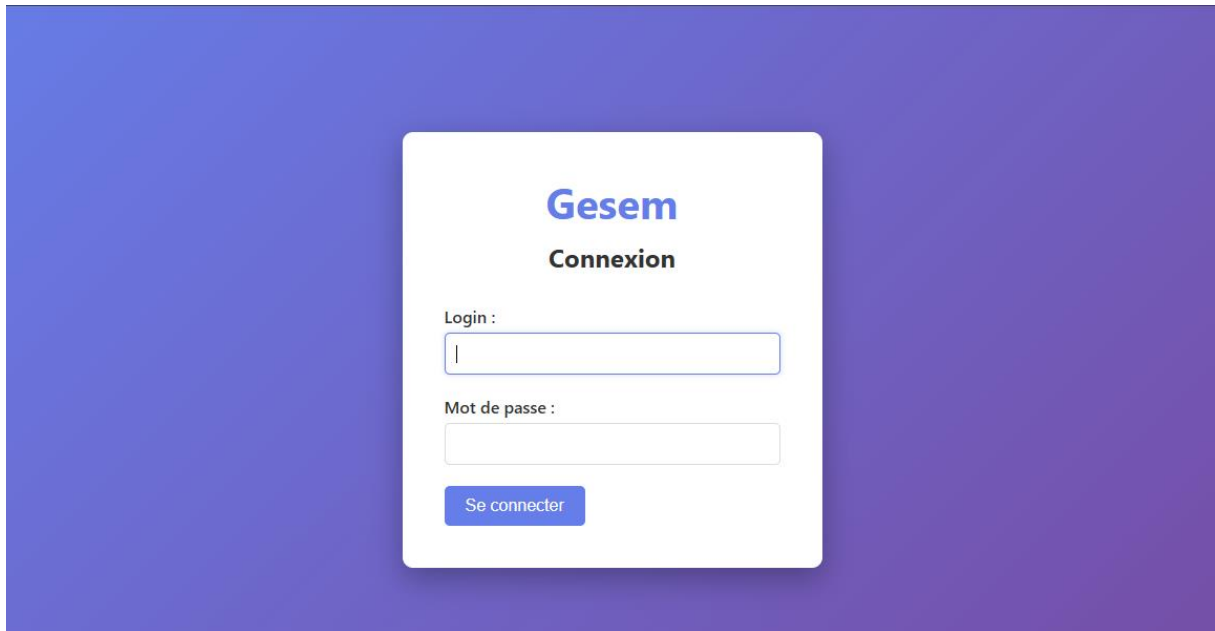


Figure 3: Page de connexion

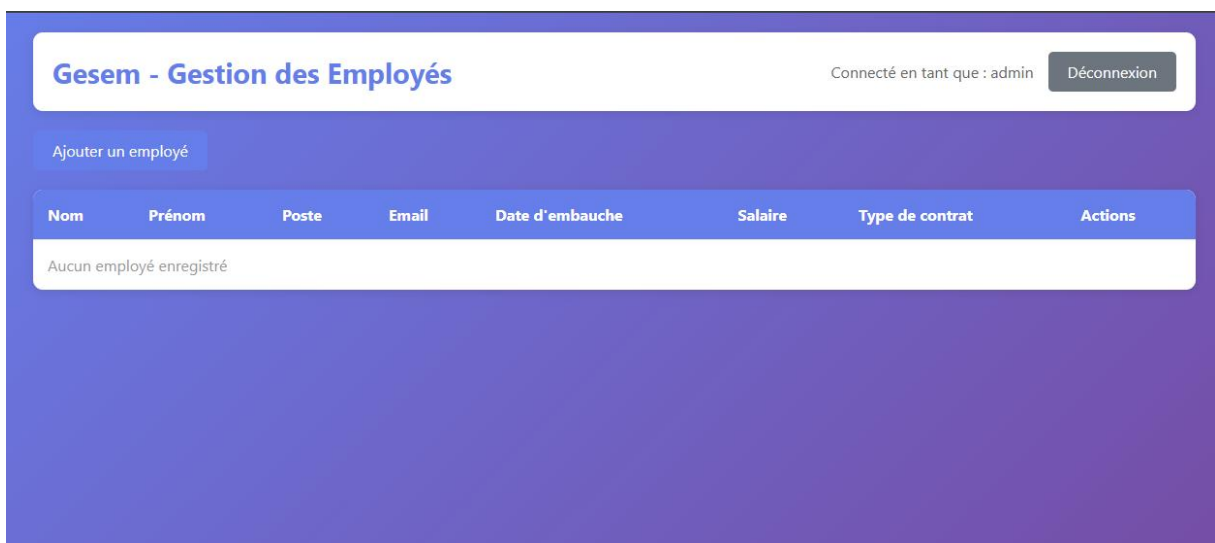


Figure 4: Page de liste des employés avec aucun employé ajouté

Gesem - Ajouter un Employé

Connecté en tant que : admin

Déconnexion

Nom * :

Prénom * :

Poste * :

Email * :

Date d'embauche * :

jj/mm/aaaa

Figure 5: Page d'ajout d'un employé

Gesem - Gestion des Employés

Connecté en tant que : admin

Déconnexion

Employé ajouté avec succès

Ajouter un employé

Nom	Prénom	Poste	Email	Date d'embauche	Salaire	Type de contrat	Actions
Traore	Drissa Sidiki	Developpeur	drissasidiki7219@gmail.com	28/10/2025	1 000 000,00 €	CDI	Modifier Supprimer

Figure 6: Page de liste des employés avec un nouvel employé ajouté

Gesem - Modifier un Employé

Connecté en tant que : admin

Déconnexion

Nom * :

Sidibe

Prénom * :

Drissa Sidiki

Poste * :

Developpeur

Email * :

drissasidiki7219@gmail.com

Date d'embauche * :

28/10/2025

Salaire * :

Figure 7: Page de modification de l'employé précédemment ajouté

Gesem - Gestion des Employés

Connecté en tant que : admin

Déconnexion

Employé modifié avec succès

Ajouter un employé

Nom	Prénom	Poste	Email	Date d'embauche	Salaire	Type de contrat	Actions
Sidibe	Drissa Sidiki	Developpeur	drissasidiki7219@gmail.com	28/10/2025	1 000 000,00 €	CDI	ModifierSupprimer

Figure 8: Page de liste des employés avec l'employé modifier

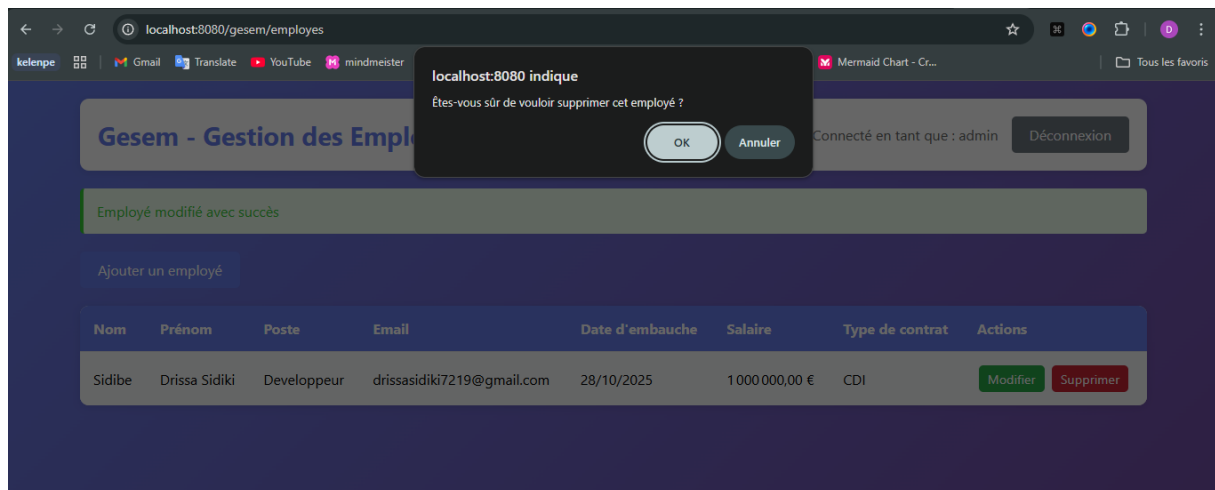


Figure 9: Demande de confirmation de suppression d'un employé

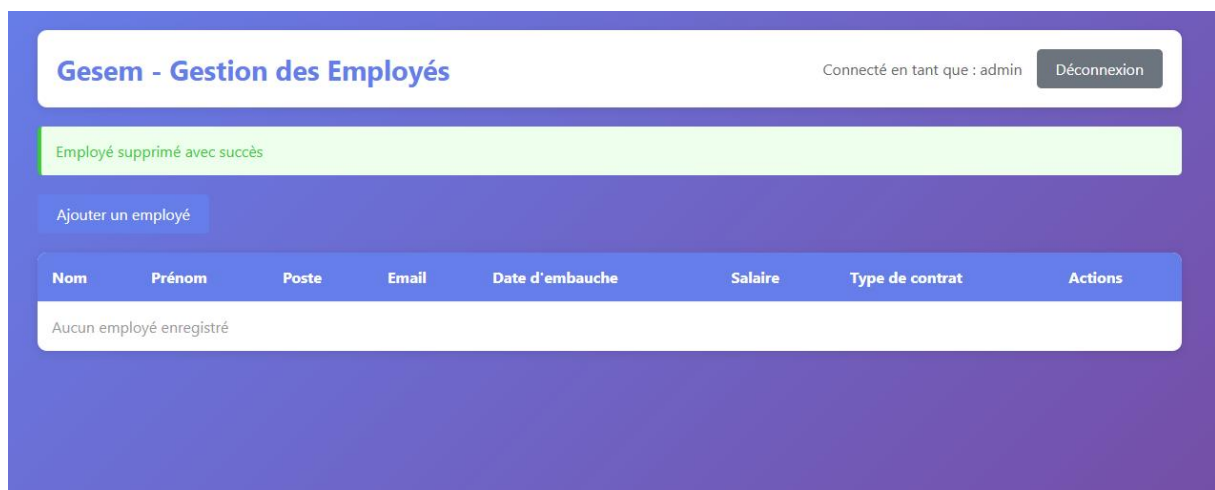


Figure 10: Page de liste des employés avec l'employé supprimé avec succès

Conclusion

Bilan du projet

Le projet **GESEM** a été mené à bien et atteint pleinement les objectifs fixés.

Toutes les fonctionnalités principales ont été implémentées avec succès :

- **Authentification complète** avec gestion de session (login/logout).
- **CRUD fonctionnel** permettant la gestion intégrale des employés.
- **Architecture MVC** assurant une séparation claire entre les couches de l'application.
- **Persistence JPA** intégrée à une base de données **PostgreSQL**.
- **Sécurité de base** via des filtres de protection des ressources.

Sur le plan technique, ce projet a permis de renforcer plusieurs compétences clés :

- Maîtrise des **Servlets** et **JSP** pour la création d'interfaces dynamiques.
- Utilisation de **JPA** pour la gestion de la persistance des données.
- Compréhension approfondie de l'**architecture MVC**.
- Configuration et déploiement d'une application **Jakarta EE** complète.
- Mise en place d'une gestion de **session** et de **sécurité web**.

Perspectives d'évolution

Plusieurs axes d'amélioration sont envisageables pour faire évoluer le projet :

1. Renforcement de la sécurité

- Hashage des mots de passe (ex. : **BCrypt**).
- Protection **CSRF** et usage obligatoire de **HTTPS**.
- Gestion avancée des rôles utilisateurs (admin, user).

2. Nouvelles fonctionnalités

- Ajout d'un module de **recherche et filtrage**.
- Exportation des données au format **PDF** ou **Excel**.
- Mise en place de la **pagination** pour les grandes listes.
- Suivi de l'**historique des modifications**.

3. Évolution de l'interface utilisateur

- Adoption d'un framework CSS moderne (**Bootstrap**, **TailwindCSS**).
- Validation des formulaires côté client via **JavaScript**.
- Conception d'une **interface responsive** adaptée aux mobiles.

Apports personnels

La réalisation de ce projet a constitué une expérience formatrice et concrète, permettant de :

- Assimiler les **fondamentaux de Jakarta EE** (Servlets, JSP, JPA).
- Mettre en œuvre de manière pratique le **modèle MVC**.
- Gérer efficacement la **persistance des données** avec JPA et un SGBD relationnel.

- Développer une **application web complète**, du modèle jusqu'à l'interface utilisateur.
- Comprendre et appliquer les **principes de sécurité web** (authentification, filtrage).

Ces acquis constituent une base solide pour évoluer dans le **développement d'applications Java Enterprise**, et offrent une véritable valeur ajoutée dans un contexte professionnel.